8
Secret Agents

In practice, AI automation will be the predominant use case of AI in the long term, more so than generation.

Every organization wants to go bigger, move faster, get it done cheaper, and reduce churn inflicted by human resources, infrastructure, and software costs. Effectively, the key result for any objective is to maximize margins in the returns on investments. While producing high-quality products is often cited as the goal, the end goal is simply to get the most out of consumers while exerting the least amount of effort to get it. People think AI will displace or even replace people, but it is actually automation. This has been occurring over many generations. It's not new. AI is just the current means, the next wave in an ever-endless cycle of bigger, faster, cheaper.

We often talk about how automation systematically handles repetitive, mundane, and error-prone tasks reliably. However, in past cases, automation was simply a substitution of resources. Though automation proved useful for solving certain repetitive tasks and reducing errors, we still suffer from many of the same afflictions. In essence, we just shifted the problem without actually solving it. Hidden costs caused by cultural and process problems make measurable gains from automation quite elusive. Additionally, most organizations don't know how to navigate technical debt responsibly. As a result, mileage covered by automation will vary significantly by organizational structure, use case, skillset, and experience. The important thing to remember is that automation only helps if you are traveling in the right direction to begin with. Otherwise, you are just traveling in the wrong direction faster, and your problems definitely will be bigger. It will also be much more expensive to come back.

While the quest to automate everything isn't anything new, the approach using AI is different and promises to breathe new life into the problem. Given proper constraints, an "intelligent" system might actually be able to adapt to changing conditions in real time. This is key. Only in uncertain cases that require a high degree of accuracy will human oversight be needed.

The **Model Context Protocol** (**MCP**) is arguably the most essential property of any agentic system since it allows AI agents to communicate with external systems in a standard way. This is necessary in order for AI agents to acquire the necessary context used to solve problems. Being able to pull context from external data sources and disparate systems at the precise moment when the data is needed and used will improve the accuracy of the actions performed by the AI agent.

In this chapter, we learn about AI agents along with how to create them using Spring AI to do useful tasks. We will also look at Embabel, pronounced Em-BAY-bel. Embabel is a new JVM framework created by the father and creator of Spring himself, Dr. Rod Johnson. Embabel is built on Spring AI and is used to develop Agentic AI applications.

In this chapter, we will also explore other AI facilities used in developing AI applications, such as the MCP. We will revisit tool calling, which we briefly touched on in *Chapter 2*.

In this chapter, you will learn:

- How to compose agentic workflows
- How to use Spring AI to build AI agents
- How to connect an AI agent to an external data source using MCP?
- How to use an AI agent as your assistant to automate simple tasks

These lessons are broken down into four main sections, starting with:

- Overview of Agentic AI
- Building AI Agents with Spring AI
- Invoking Spring AI Agents using an MCP Client
- Building AI Agents with Embabel, Koog and LangChain4j

Let's get to it.

Technical requirements

The examples presented in this chapter are based on Java 17 and Spring Boot 3.5.x. The examples use Spring AI 1.0.x.

You must manually install JDK 17 or later. The project Maven POM declares and resolves Spring AI, Spring Boot, and all required dependencies from Maven Central. You can find the source code for the examples in the GitHub Repository:

https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI

Additionally, you will need to sign up and register for an API key at SerpApi. You can register at serpapi.com/users/sign_up. The use of SerpApi was necessary to interface with the travel provider called by the Spring AI Agent.

You will also need to install an AI desktop application that can understand the MCP. In the example below, we use Anthropic's Claude Desktop to demonstrate the Spring AI Agent in action. You can download and install Claude Desktop for your system from claude.com/download. If you need help, refer to the support page at support.claude.com/en/articles/10065433-install-claude-desktop.

It is also possible to use other compatible MCP clients and AI provider applications, such as OpenAI ChatGPT, but I will not cover how to use those tools in this chapter.

Overview of agentic AI

Agentic AI encompasses all aspects of AI agents. Loosely speaking, an AI agent is any autonomous system or process that can make decisions and perform actions over an extended period of time without human intervention. The degree of control afforded to an AI agent varies by use case. An AI agent can be fully autonomous and simply output the results of its decisions and actions, or an AI agent can perform pre-defined actions following a closely orchestrated workflow that includes checks and balances.

One example of a fully autonomous agent might be a process that regulates the temperature of your home based on time of day and weather conditions. This type of agent could be allowed to adjust the temperature of your home or office within a pre-defined temperature range. An example of a controlled agent might be to serve as a travel assistant that helps plan and coordinate a trip. The agent could book a flight, reserve a hotel, and rent a car on your behalf. However, at each step in the process, the agent might confirm the time of day for the flight, whether to book two queens or a single king-size bed, and inquire about the Ferrari upgrade since the rental company is running a special for only $2000 USD per day.

In a publication posted by Anthropic in December of 2024, Anthropic summarized findings from research they conducted with customers across various industries building AI agents. See https://www.anthropic.com/engineering/building-effective-agents. They classified AI agents into two distinct categories:

- In **workflow-based systems,** LLMs perform tasks using prescriptive implementations with predefined workflows and rule-based code paths.
- In **agent-based systems,** LLMs decide which processes and tools to execute. In this case, the AI agent is self-directing, controlling what and how tasks get accomplished.

In either case, Anthropic recommends starting with the simplest approach possible, then adding additional complexity only when strictly needed. Added complexity should be organized in a structured manner taking into account both observable and secure measures throughout processing. As autonomous systems become more self-directing, having a trace log for everything that happened and why will be essential for auditing purposes. In some cases, it may even be required for legal or regulatory compliance.

Given the need to maintain oversight, it is not that surprising to find that workflow-based systems tend to be more common in practice. After all, rule-based, prescriptive systems predate practical applications of AI by several decades. However, you should expect that as AI models improve in both reasoning and capabilities, AI will become an ever-increasing and integral part of workflow-based processing. Advances in AI have already led to several AI-integrated workflow-based processing pipelines. These pipelines are categorized as follows:

- **Chained-sequential workflow (aka prompt chaining)**: Complex tasks are broken into a sequence of manageable steps where output from the previous LLM is used as input to the next LLM. An agentic sequential workflow is depicted in the

following diagram.

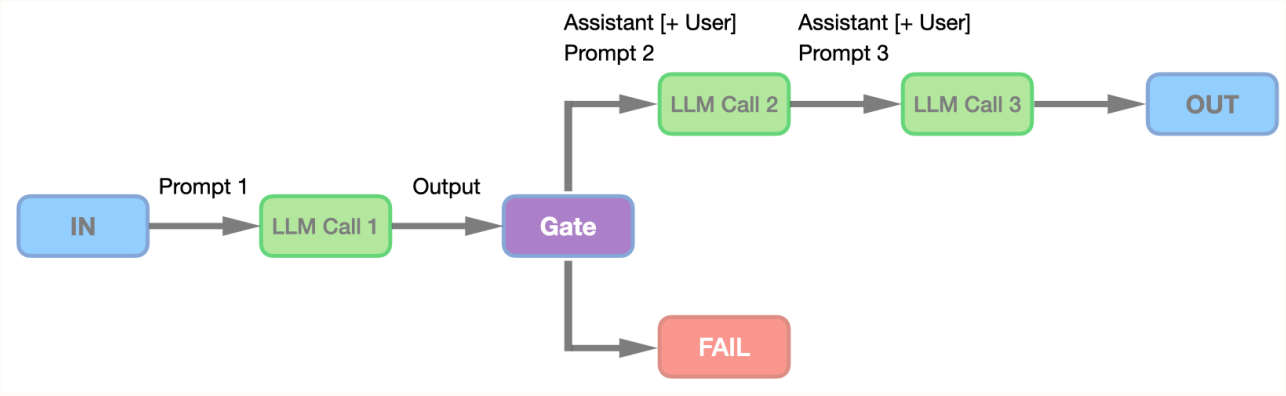


Figure 8.1: Sequential Workflow

- **Parallelization workflow:** Runs independent or intermediate tasks in parallel, then aggregates the results to produce the final solution. In sorting algorithms, the most comparable example is the divide and conquer method of the Quick Sort algorithm. An agentic parallelization workflow is depicted in the following diagram.

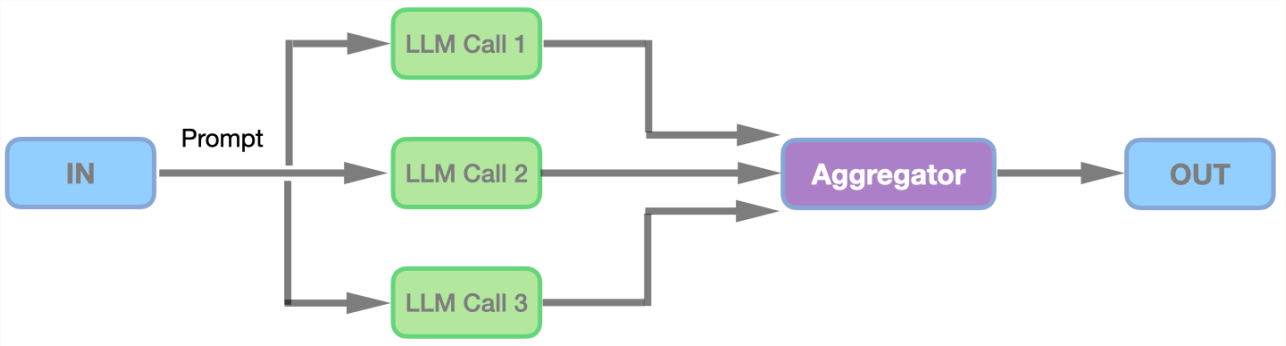


Figure 8.2: Parallelization Workflow

- **Routing workflow**: Optimizes task execution based on input classification. After properly evaluating and classifying input, carefully crafted prompts can be sent to more specialized LLMs specifically trained for the given task thereby resulting in higher quality output. A general LLM call can be used to evaluate input and route requests. An agentic routing workflow is depicted in the following diagram:

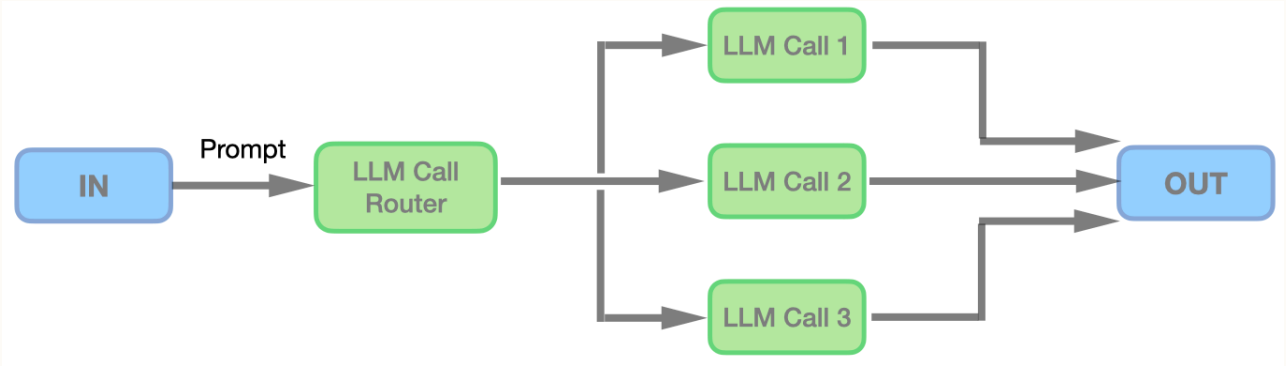


Figure 8.3: Routing Workflow

- **Orchestrator workflow**: Much like the parallelization and routing workflows, the orchestrator uses a central LLM that dynamically splits a task into subtasks determined at runtime, routing to an appropriate worker LLM executing in parallel. After subtasks have successfully completed, the orchestrator synthesizes the results and generates a final solution. An agentic orchestrating workflow is depicted in the following diagram:

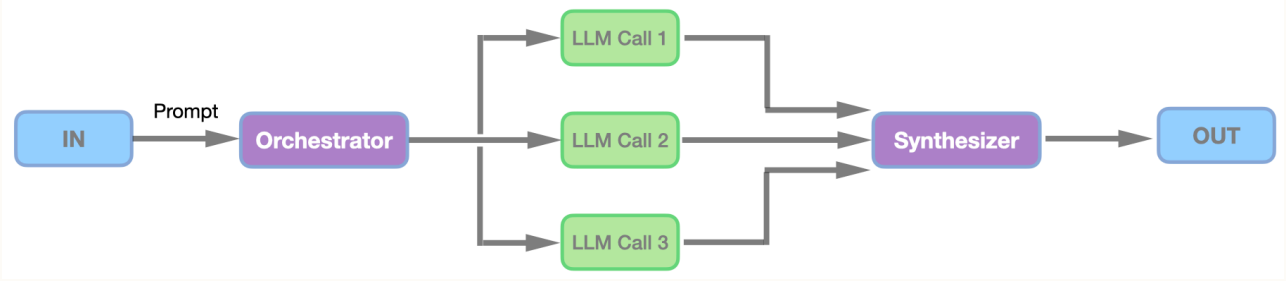


Figure 8.4: Orchestrator Workflow

- **Evaluator-Optimizer workflow**: This workflow is effective when evaluation criteria are clear, objective and measurable. Intermediate solutions can be generated and iteratively refined until the end result passes the defined acceptance criteria, satisfying the "*definition of done*". The specialized LLM generates a solution while a second LLM evaluates the results and either accepts the solution or rejects the generated result and provides feedback. An agentic evaluator-optimizer based workflow is depicted in the following diagram.

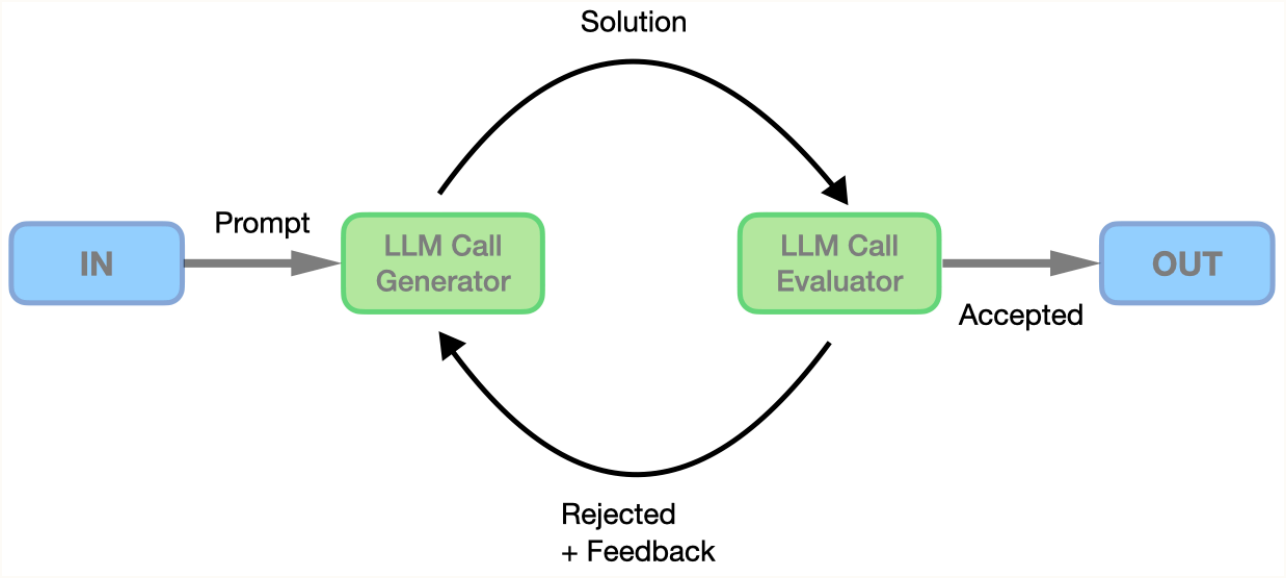


Figure 8.5: Evaluator-Optimizer Workflow

As you might imagine, other types of workflows do exist. However, these happen to be the most commonly used workflows in practice. Also, multiple workflows could be combined at different stages in a processing pipeline to further create more sophisticated workflows with different degrees of accuracy and quality guarantees.

For example, perhaps certain tasks require a combination of an **evaluator-optimizer** workflow coordinated by an **orchestrator**, as depicted in the following diagram:

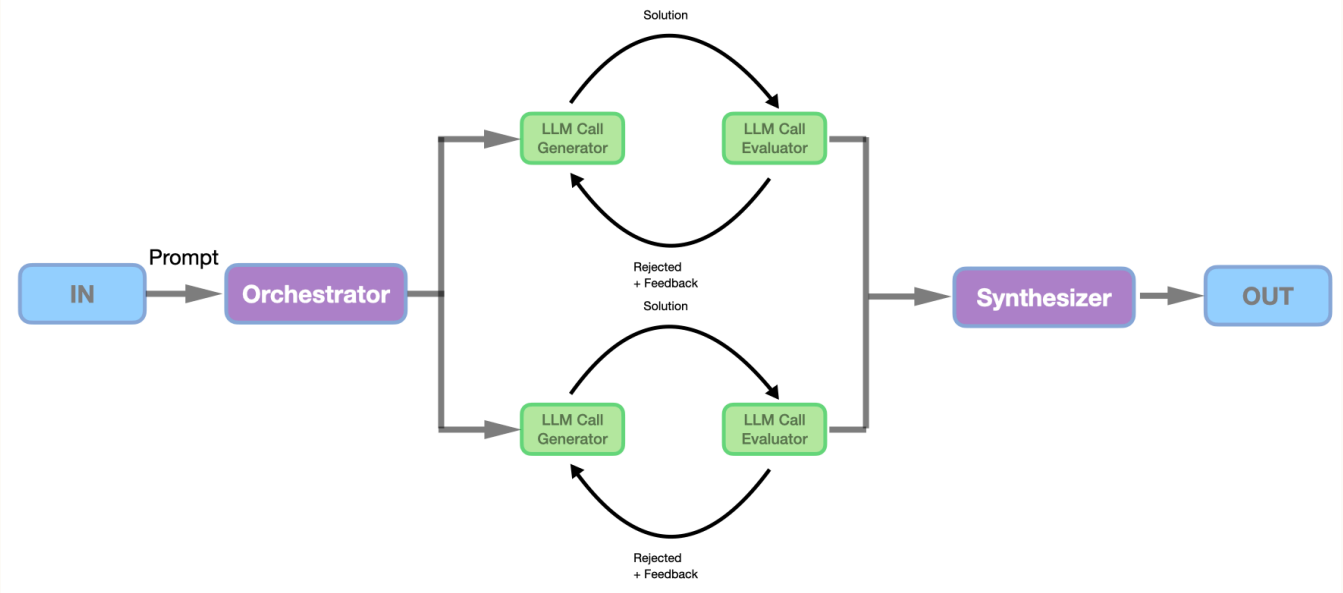


Figure 8.6: Orchestrator + Evaluator-Optimizer Workflow

Whichever workflow or combination of workflows you choose, it is important to take into consideration throughput and latency. Oftentimes, you will be trading lower throughput and higher latency for better accuracy. As with everything, there are always trade-offs. Typically, your use case, requirements, and SLAs will be the deciding factor.

Let's quickly summarize the workflow options in the following table:

Workflow	Description
Sequential Workflow	Breaks a complex task into a set of related and dependent subtasks processed in sequential order. Output from the previous task is fed into the input of the next task.
Parallel Workflow	Runs independent and intermediate tasks in parallel, aggregating the results to produce the final solution.
Routing Workflow	Uses input classification to decide on the appropriate task execution. Tasks are routed to specialized LLMs trained for specific tasks in order to produce higher-quality output.
Orchestrator Workflow	Uses a central LLM to dynamically split a complex task into subtasks determined at runtime and then delegates the computation to worker LLMs executing in parallel.
Evaluator-Optimizer Workflow	Uses a specialized LLM to generate a solution and a second LLM to evaluate the output, iterating until a desired outcome is achieved. Useful when evaluation criteria are clear, objective, and measurable.

Table 8.1: Agent Workflow Patterns

In any agentic system, the main components constituting an AI agent are similar. For an AI agent to perform effectively, it needs adequate context. Training alone is not enough. Training only provides the foundation for reasoning, but contextually relevant and current information is a key differentiator for affecting the quality of the generated output. Given an "informed" view of the world, an AI agent is equipped to make better decisions and take the appropriate action. Augmentation is the key component of RAG. By further decomposing the problem into subtasks, the workflow can be simplified. At each step, memorizing what happened before and what should happen next, especially where timing is involved, is also essential to changing course.

RAG, Tool Calling, and Memory are all essential components of an agentic system that enables AI agents to achieve the desired outcome. With any system, tuning is required. The typical agentic system is depicted in the following diagram:

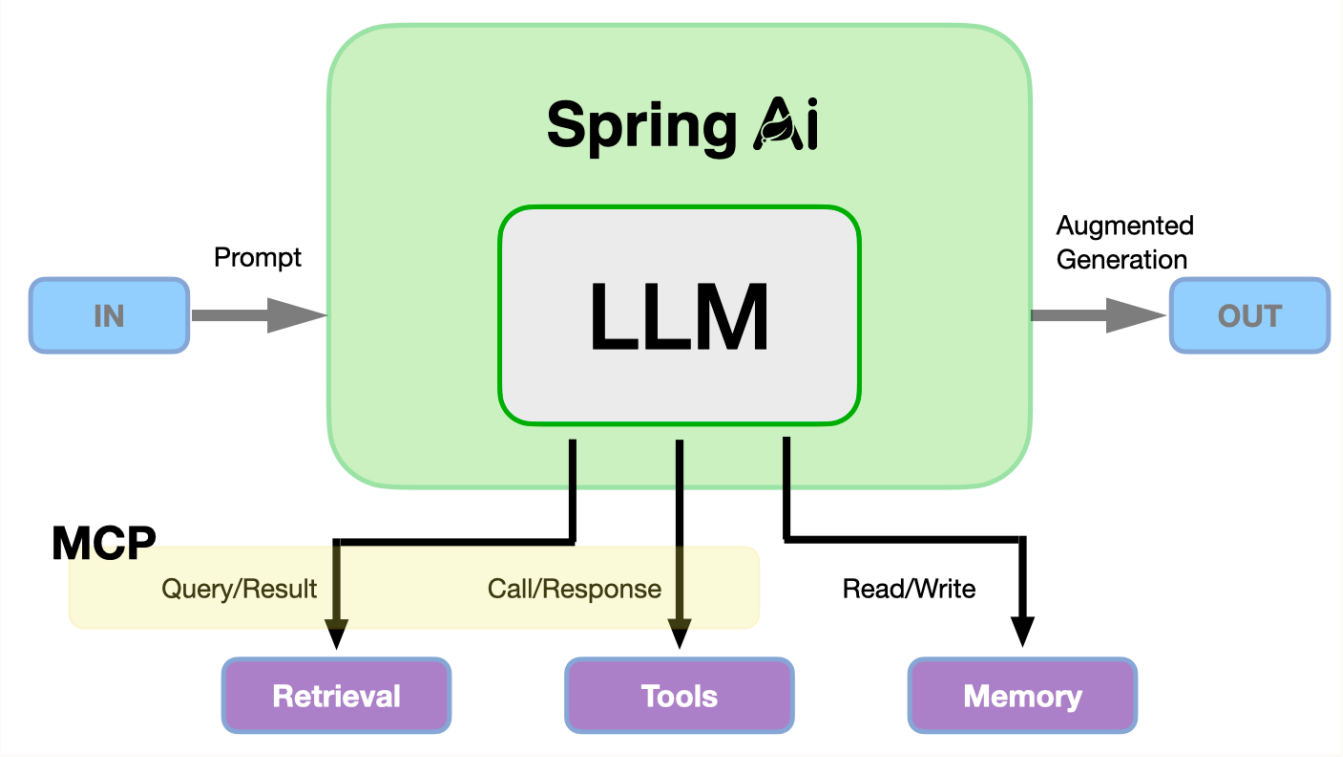


Figure 8.7: Anatomy of an Agentic System using Spring AI

As implied in the diagram above, the AI agent can be built as a Spring Boot application using key features of Spring AI. As we have learned in previous chapters, Spring AI includes comprehensive support for RAG, Tool Calling and Chat Memory. Spring AI's APIs and modular architecture provide first-class constructs for all 3 aspects of an agentic system.

Spring AI, perhaps with the help of Spring Data, enables additional context to be retrieved from any external data source, which can be used to augment the prompt. In turn, an AI model can make better, informed decisions about the appropriate actions to take. The actions can be expressed in Tool calls or function callbacks invoked by an AI model. Predefined functions offer more control over what can and cannot be done. During processing in a coordinated workflow, Chat Memory can be used to remember conversational state or intermediate results between actions or tasks. Depending on the backing store, Chat Memory can even be written to disk for auditing purposes.

Additionally, Spring AI provides robust support for the MCP. MCP uses a standardized format of communication essential for connecting LLMs with external systems containing a wealth of invaluable information. Any provider can implement the protocol for their system, such as a database, file system, search engine, or whatever. The important point is that MCP enables an AI model to connect to and communicate with any MCP-compliant external data source.

> TIP
>
> You can learn more about the MCP, from the source at modelcontextprotocol.io/docs/getting-started/intro. Anthropic is the creator of MCP, which was driven by the need to connect LLMs to diverse data sources in a standardized in order to gain invaluable insights for context. Anthropic open-sourced and announced MCP in November 2024. See anthropic.com/news/model-context-protocol.

Given Spring AI's comprehensive, robust, and well-rounded support for building agentic systems, there really is no limit to what you can do.

Now, let's use the components of an agentic system to build an AI agent using Spring AI. We will demonstrate how someone might build a fully automated travel agent-based system.

Let's dive in.

Building AI agents with Spring AI

In this section, we look at the components of an agentic AI system by building an automated travel agent using Spring AI. The system architecture will consist of two components:

- Spring Boot, Spring AI application providing Tools via an MCP server

- MCP client (e.g. Claude Desktop)

The intent of the Spring Boot, Spring AI application, is to allow users to make travel arrangements with the assistance of an AI agent. In particular, the user will be able to search for flights, find available hotels, and optionally arrange for ground transportation using natural language preferences. While the system won't allow users to book the actual reservation, it would be a simple matter to implement this feature using the same tool calling logic. This is left as an exercise for the ambitious reader.

> TIP
>
> If you continue to implement a complete agentic-based travel agent, there are several important factors to consider. A few factors include, but are not limited to: security, including both auth and secure payments; transactional boundaries when coordinating travel arrangements; error handling and/or retries; and notifications, especially with changes in travel, cancellations, etc.
>
> Given this example presents available travel data as read-only, alternatively you could pass control to the providers when users are ready to book their travel arrangements. Additional downstream actions could be performed after receiving appropriate confirmations.

An MCP server represents and uses a single travel provider implementation. The travel provider can be used to make travel arrangements for major airlines, hotels, and car rentals (optional). In this example, Google will be used to implement a live travel agent for flights and hotels. Google does not provide a means to rent a car that I am aware of. Arranging a car or vehicle rental in general will be left as an exercise for the reader. A good provider option for car rentals would be Booking.com.

From a Web browser, Google Flights is available at and Google Hotels is available from . When you visit the pages, they will appear as shown below:

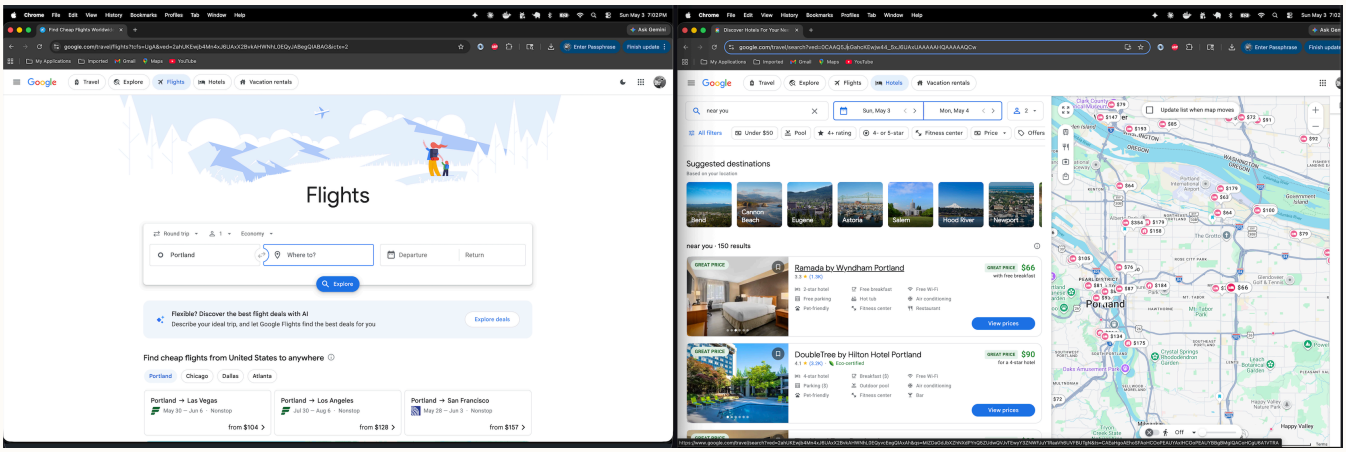


Figure 8.8: Google Flights and Google Hotels

Unfortunately, Google does not provide an API to access either Google Flights or Google Hotels programmatically. However, this does not mean it is not possible. Ironically, a quick Google Search reveals that both Google Flights and Google Hotels are accessible from SerpApi.

> You can learn more about SerpApi at serpapi.com. SerpApi provides many robust integrations with service providers, like Google. It is very impressive. You can find a complete list of search engines and provided APIs at serpapi.com/search-engine-apis.
>
> Just remember, Google can change their functional offerings at any point. SerpApi could choose to stop providing certain APIs. The SPI design in this case protects you from such changes.

There are other travel providers who offer robust APIs and SDKs as well, although I haven't explored them in detail.

Booking.com's Demand API, documented at, developers.booking.com/demand/docs, looks promising. This API can be used to search and book accommodations (e.g. hotels) as well as car rentals. I tried Expedia, but was rejected when I requested API access. Companies like Expedia and Travelocity only seem interested in integrating with "partners". Honestly, I was less than impressed with their API documentation (very poor), but their Java SDK was fairly intuitive.

In any case, the Spring Boot, Spring AI Tool-based MCP server application uses a Service Provider Interface (SPI) to easily swap between different travel providers. To change the travel provider, the user only needs to change the Spring Profile. The central interface in the SPI is the `TravelService` interface used to interact with the travel provider, such as Google. This interface encapsulates the implementation details for all providers, thereby enabling the Agent to consistently adhere to a common abstraction and interchange providers seamlessly.

Now, let's dive a bit deeper into the implementation details of the application, starting with the `TravelService` interface. The `TravelService` interface is shown below:

```
interface TravelService {
   List<Flight> searchFlights(FlightSearchRequest request);
   List<HotelBooking> findHotels(HotelSearchRequest request);
   default List<VehicleRental> rentVehicle(VehicleSearchRequest request) {
      throw new UnsupportedOperationException("Vehicle rentals not supported");
   }
}
```

As mentioned above, the application allows a user to search for flights, find a hotel, and rent a vehicle, which are all operations provided in the `TravelService` interface shown above. By default, vehicle rentals are not supported, or rather, were not implemented, and throw an `UnsupportedOperationException` by default. This function has been left as an exercise for the reader.

Each service method accepts a request object of a given type, such as `FlightSearchRequest`. The request object defines the search criteria specified by the user, such as when searching for available flights. In the search for flight, the search criteria is the same as the criteria used by Travelocity, Expedia or any airline, such as American Airlines. You would enter the airport you are departing from, the date, and preferred time of your departure, the airport at your final destination, and the date and time of your return. You might also specify your travel class, such as Business or Premium Economy, along with your preferred airline, such as United, in which you want to travel. When using Google Flights, the search criteria are entered into the Web form shown below:

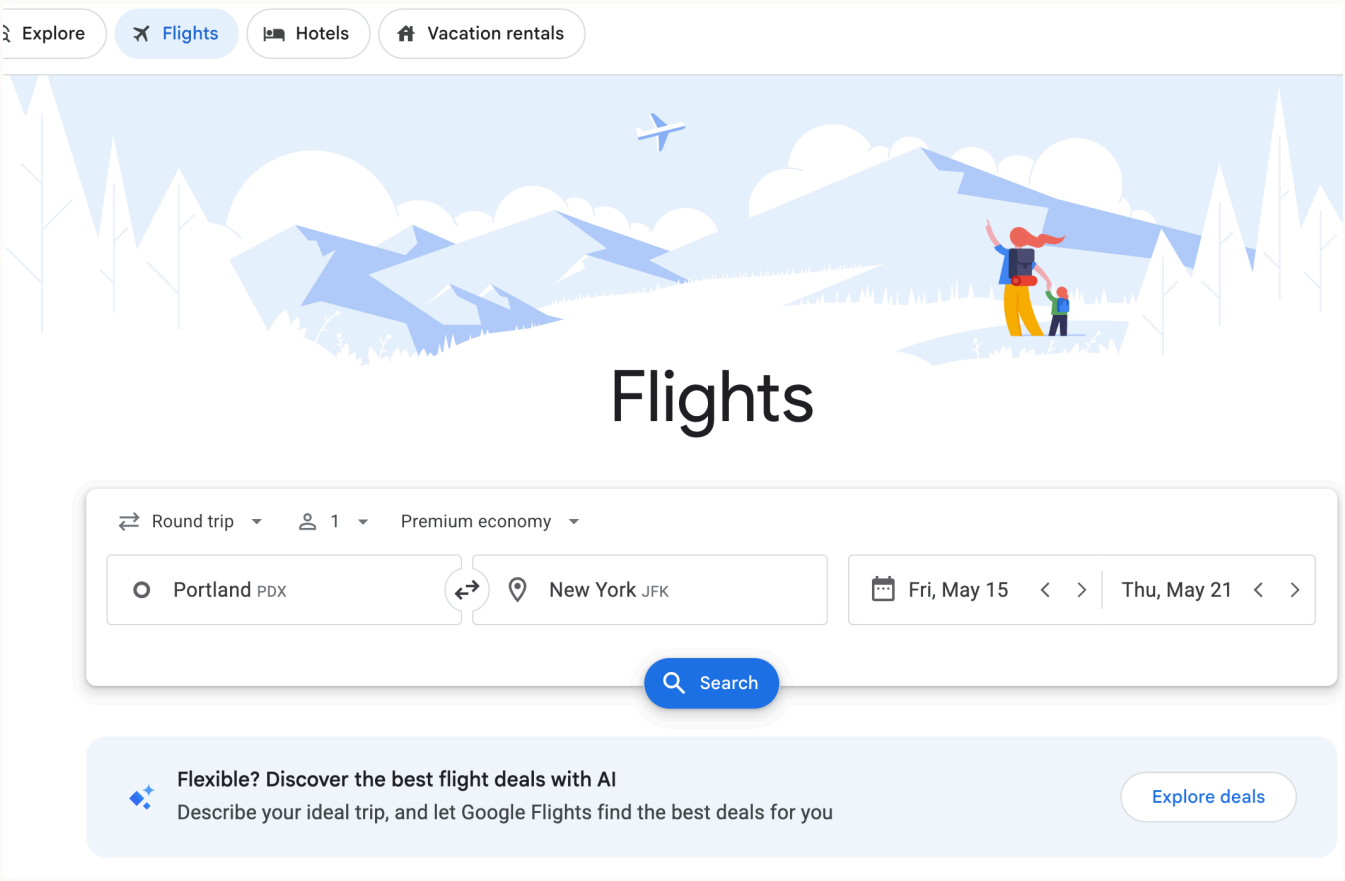


Figure 8.9: Searching for Flights using Google Flights

While it is not difficult to fill out a Web form, it would be nicer to be able to type or say our search criteria in a manner similar to talking with a live travel agent. Using an AI app, such as OpenAI's ChatGPT or Claude Desktop (shown below), you could do the following:

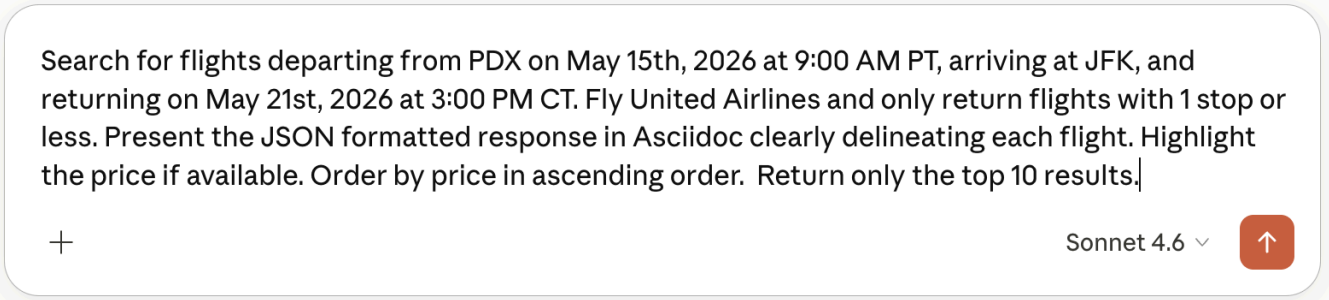


Figure 8.10: Search for Flights using Claud Desktop

Programmatically, this is precisely the search criteria that the `FlightSearchRequest` class allows you to specify using a **Domain-Specific Language** (**DSL**) dialect and syntax, as shown below:

```
Airport destinationAirport = Airport.from(arrivalAirport);
FlightSearchRequest request = FlightSearchRequest.builder()
   .departFrom(Airport.from(departureAirport))
   .departOn(departureDate.atTime(LocalTime.MIN)
      .atZone(inLocalZone()))
   .arriveAt(destinationAirport)
   .returnOn(returnDate.atTime(LocalTime.MAX)
      .atZone(inReturnZone(destinationAirport)))
   .fly(Airline.from(airlineName))
   .inPremiumEconomy()
   .stops(resolveStops(stops))
   .anyPrice()
   .build();
List<Flight> flights = getTravelService().searchFlights(request);
```

In the search criteria shown above, `departureAirport` is PDX. Date and time of departure, the `departureDate`, would be May 15th, 2026, at 9 AM PT. Arrival airport, or `destinationAirport`, is JFK. We would return on May 21st, 2026, at 3:00 PM ET. The preferred `airlineName` is United. We are flying `premiumEconomy` class with no regard to the price.

After the `FlightSearchRequest` object is built, the object is passed to the `TravelService`, `searchFlights(request)` service method to begin searching for available flights matching the search criteria. The `TravelService` SPI is a facade for a configured travel provider, which in our case is Google Flights. However, any travel provider implementing and conforming to this application interface will work.

The call blocks until Google Flights returns the results. The service method returns a `List` of `Flight` objects that effectively captures the precise details of the flight, such as the exact departure date/time, return date/time, price, and so on. The exact details of a flight are determined by the travel provider, which is typically the airline, such as United Airlines. A definition of the `Flight` type can be seen below:

```
public record Flight(
   String number,
   Departure departure,
   Arrival arrival,
   Airline airline,
   Aircraft aircraft,
   Aircraft.Seat seat,
   Duration duration,
   BigDecimal price
)
```

As you can see above, the `Flight` type is a Java record. It uses public static helper types, such as `Departure`, `Arrival`, and `Airline` to represent the internal state of a flight in an intuitive and descriptive manner. Additionally, the `Flight` record provides a `Builder` via the `builder(..)` factory method to easily construct a `Flight` in a manner similar to the `FlightSearchRequest` by using a custom Domain-Specific Language (DSL).

Both `Departure` and `Arrival` capture the departure and destination airport as well as the date/time of departure and arrival, respectively. `Departure` and Arrival are both defined as follows:

```
record Departure(Airport airport, ZonedDateTime dateTime)
record Arrival(Airport airport, ZonedDateTime dateTime)
```

`Airport` captures the airport code, such as "JFK" for John F. Kennedy International Airport in New York City, NY, USA. `Airline` captures the name of the airline, such as "United Airlines," and `Aircraft` captures the make and model of the airplane that will be piloted for the flight, such as "Boeing 747."

In addition to the basic search criteria and flight details, a `Flight` object can also capture the seat assigned during reservation, the duration of the flight, captured as a `java.time.Duration`, and the price paid for the flight. Of course, many more details could be captured, such as the number of stops and legs of the flight. The level of granularity is up to the implementor of the application and what is necessary to present to the end-user of the application.

The request and return model types for finding hotels and renting vehicles are similar. Therefore, we won't cover hotels and vehicle rentals in detail here. Instead, you can review the source code in the repository for the `agent-example`.

Now we need to enable and plug in an actual implementation of the `TravelService` SPI. The `agent-example` already provides an implementation using Google Travel, and specifically, Google Flights and Google Hotels accessed with SerpApi's provided REST APIs.

The provider implementation using Google and SerpApi has been implemented in the `GoogleTravelService class, which is available in the agent-example/google-travel-provider` module. The primary purpose of this class is to make calls to Google Flights and Google Hotels using SerpApi, and then convert the Data Transfer Objects (DTOs) into the model types, such as `Flight`, of the Travel API used by the Travel Agent to access the travel details.

The **Data Transfer Objects** (**DTOs**) are used to capture the JSON data returned by the SerpApi REST APIs for Google Flights and Google Hotels, respectively. In order to make the invocation of SerpApi's REST APIs for Google Flights and Google Hotels as simple as possible, I leverage the core Spring Framework's REST Clients. In particular, the example makes use of Spring's HTTP Service Clients.

> You can learn more about Spring Framework REST Clients in the reference documentation at docs.spring.io/spring-framework/reference/integration/rest-clients.html. You can learn more about HTTP Service Clients in the docs at docs.spring.io/spring-framework/reference/integration/rest-clients.html#rest-http-service-client. If you are familiar with Retrofit, square.github.io/retrofit/, Spring's HTTP Service Clients are similar, but are Spring, and therefore, are highly configurable and extensible, following the Spring programming model.

In a nutshell, when using Spring's HTTP Service Clients support, I define a simple Java interface that specifies the contract of the SerpApi REST APIs. I do this for both Google Flights and Google Hotels, as shown below:

```
@HttpExchange(accept = "application/json")
interface GoogleFlightsApi {
  @GetExchange
  FlightSearchResults search(FlightSearchQuery searchQuery);
}

@HttpExchange(accept = "application/json")
interface GoogleHotelsApi {
  @GetExchange
```

```
    HotelSearchResults search(HotelSearchQuery searchQuery);
}
```

The query and result types, such as `FlightSearchQuery` and the `FlightSearchResults` for Google Flights search and the `HotelSearchQuery` and `HotelSearchResults` for Google Hotels search, are the DTOs, and are converted to/from JSON when invoking the REST APIs. The Spring @GetExchange annotation specifies that the call performs a GET HTTP request. Jackson's Databind annotations were used to specify the mapping between the JSON and the POJOs, or DTOs.

The specification, along with examples of using SerpApi's REST API for Google Flights, can be found at serpapi.com/google-flights-api. And the specification and examples of using SerpApi's REST API for Google Hotels can be found at serpapi.com/google-hotels-api. You can see the JSON mapping defined in the query and result DTOs inside the `google-travel-provider` module. While the JSON mapping is rather straightforward, it is too lengthy to cover here. Readers familiar with Jackson data binding (and mapping) using the annotation model will already be familiar with this approach. It is not Spring-specific, but widely used across the Spring portfolio.

The `GoogleFlightsApi` and `GoogleHotelsApi` HTTP Service Clients shown above are then injected into the `GoogleTravelService` class and used to call SerpApi's REST API.

The implementation of the flight search using Google's travel services can be seen below:

```
@Service
class GoogleTravelService implements TravelService {
    @Override
    List<Flight> searchFlights(FlightSearchRequest request) {
        FlightSearchQuery query = FlightSearchQuery.from(request);
        FlightSearchResults results = getGoogleFlights().search(query);
        return results.toFlights();
    }
}
```

The implementation of the hotel search using the Google travel service is similar to the flight search and is seen below:

```
@Service
class GoogleTravelService implements TravelService {

    @Override
    List<HotelBooking> findHotels(HotelSearchRequest request) {
        HotelSearchQuery query = HotelSearchQuery.from(request);
        HotelSearchResults results = getGoogleHotels().search(query);
        return results.toHotelBookings();
    }
}
```

By using the REST API interfaces and DTOs to convert between the Travel API model types, such as the request types and return types in the service method signatures shown above, the `GoogleTravelService` implementation can be kept quite simple.

> TIP
>
> The Spring @Service GoogleTravelService class service methods are an ideal *join point* to introduce one or more Spring AOP Aspects that handle cross-cutting concerns, like retries in case of API timeouts or transient network errors using Spring Retry. Alternatively, an AOP Aspect can be used to rate-limit SerpApi access and prevent DDOS attacks. Additionally, a Circuit Breaker can be applied to prevent cascading failures in the case where SerpApi is unavailable, or certain 5xx HTTP errors might occur.

Now that we have defined our Travel SPI and implemented a provider using Google Travel, we are well-positioned to implement the Travel Agent.

The Spring AI Travel Agent is simply a Java class and POJO, declared as a Spring bean in the Spring application context of the Spring Boot application. The `TravelAgent` class effectively defines the Tool calls of the Agent that will be invoked by an AI

model. The `TravelAgent` class effectively defines the interface between the AI model and the `TravelService`, such as the `GoogleTravelService` implementation. The Travel Agent and AI model communicate using the MCP.

Once again, you can learn more about the MCP from the source at modelcontextprotocol.io.

A partial implementation of the `TravelAgent` can be seen as follows:

```
class TravelAgent {
   Environment environment;
   TravelService travelService;
   // Tool call definitions
}
```

The Spring `Environment` and `TravelService` are both injected into the `TravelAgent` to perform the functions of the Agent when called upon by the AI model.

The signature of our Tool call used when searching for flights appears as follows:

```
@Tool(name = "flight-search", description = "Search for flights")
List<Flight> searchFlights(
   @ToolParam(description = "Departure airport") String departureAirport,
   @ToolParam(description = "Departure date/time in format yyyy-MM-dd, ignore time, for example 2026-05-21") LocalDate departureDate,
   @ToolParam(description = "Arrival airport") String arrivalAirport,
   @ToolParam(description = "Return date/time in format yyyy-MM-dd, ignore time, for example 2026-06-02") LocalDate returnDate,
   //...
) {
   // Tool call implementation
}
```

Spring AI's `@Tool` annotation declares an (MCP) Tool, named "*flight-search*", and makes it known to the AI model and available to call. Of course, no Tool call would be complete unless an AI model could also pass arguments to the parameters, as defined by the `@ToolParam` annotated `searchFlights(..)` method parameters, such as `departureAirport`.

You are even allowed to use complex types, such as `java.time.LocalDate`, for arguments like `departureDate`. You just need to instruct the AI model how to pass the argument, which is specified in the `@ToolParam`'s `description` attribute as: "*Departure date/time in format yyyy-MM-dd, ignore time, for example 2026-05-21*". Spring AI will take care of converting the String formatted date passed by the AI model to the "*flight-search*" Tool as a `java.time.LocalDate` on invocation.

The actual Tool call implementation is defined as:

```
Airport destinationAirport = Airport.from(arrivalAirport);
FlightSearchRequest request = FlightSearchRequest.builder()
   .departFrom(Airport.from(departureAirport))
   .departOn(departureDate.atTime(LocalTime.MIN).atZone(inLocalZone()))
   .arriveAt(destinationAirport)
   .returnOn(returnDate.atTime(LocalTime.MAX).atZone(inReturnZone(destinationAirport)))
   .fly(Airline.from(airlineName))
   // ...
   .build();
List<Flight> flights = getTravelService().searchFlights(request);
return flights;
```

As shown above, the *"flight-search"* Tool call is translated into a `TravelService` call. First, a Travel API FlightSearchRequest object is constructed from the arguments passed to the Tool call. The request is then passed to the `searchFlights(..)` service method to begin the search for available flights using the Google Travel, Google Flights service provider implementation.

Once the `List` of `Flights` are returned, they are simply passed back to the AI model as JSON. You can instruct the AI model to format the JSON Tool call response however you like, as we will see momentarily.

Similarly, the "hotel-finder" Tool call is much like the "flight-search" Tool call in both form and function.

The specification for the `findHotels(..)` method is shown below:

```
@Tool(name = "hotel-finder", description = "Find a hotel")
List<HotelBooking> findHotels(
   @ToolParam(description = "Name of the hotel to search") String hotelName,
   @ToolParam(description = "Check-in date") LocalDate checkIn,
   @ToolParam(description = "Checkout date") LocalDate checkout,
   @ToolParam(description = "Number of occupants") Integer occupants
) {
   // Tool call implementation
}
```

The `findHotels(..)` method is then defined as:

```
HotelSearchRequest request =
   HotelSearchRequest.stayAt(Hotel.from(hotelName))
      .checkIn(checkIn.atStartOfDay(inLocalZone()))
      .checkout(checkout.atTime(MAX).atZone(inLocalZone()))
      .payAny()
      .withOccupants(occupants)
      .build();
List<HotelBooking> hotelBookings = getTravelService().findHotels(request);
return hotelBookings;
```

Just like "*flight-search*", the "*hotel-finder*" Tool call captures arguments passed by the AI model, constructs a `HotelFlightRequest` and invokes the `TravelService, findHotels(..)` service method to search for available hotels. All that is left now is to enable the MCP Server as part of the Spring Boot application.

Spring AI makes the task of enabling a Spring Boot application to serve as an MCP server about as simple as possible. It is pretty darn cool! Simply declare the `spring-ai-starter-mcp-server` dependency in your project's Maven POM dependencies and you are done!

```
<dependency>
   <groupId>org.springframework.ai</groupId>
   <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
```

You also have the option to declare a Web-based MCP server that wraps MCP in an HTTP transport by declaring the `spring-ai-starter-mcp-server-webmvc` dependency. Of course, Spring Web MVC is blocking, so, alternatively, you also have the option to use Spring WebFlux. Simply declare the `spring-ai-starter-mcp-server-webflux` dependency instead. Spring WebFlux uses Project Reactor under the hood to make all AI Tool call invocations reactive, or non-blocking with back pressure. This is ideal in async use cases where responsiveness and compute resource utilization is critical.

> If your travel agent application is set up as a Web application hosted remotely, such as in the cloud running in AWS, GCP or Azure, then you must use Spring Web MVC or WebFlux MCP server starters. STDIO is not an option. Additionally, it is recommended to enable auth between your MCP server and client.

The last bit of configuration is simply to register for an API key to make REST API calls to SerpApi. You can register for an API key at serpapi.com/users/sign_up. After you have registered and signed up, navigate to the "Your Account" dashboard. There, you will see your API key, as shown below:

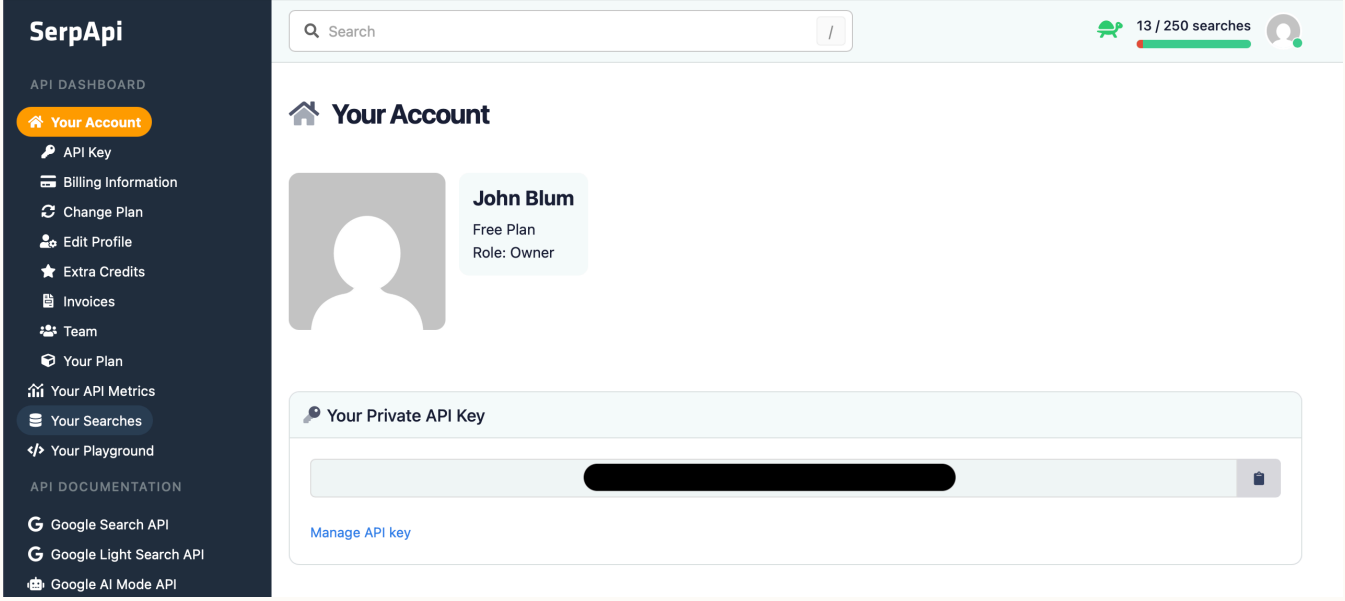


Figure 8.11: SerpApi User Account Dashboard

You will also find other useful information on the Account Dashboard.

After you acquire your API key, set the following property in the `application-user.properties` file, as seen below:

```
travel-agent.app.provider.serpapi.api-key=<your-api-key>
```

> **NOTE**
>
> The `application-user.properties` file is ignored by the `spring-ai-examples` project. Therefore, you can be rest assured that you won't incidentally check in your SerpApi API key to your cloned `spring-ai-examples` GitHub repository.

Now, we are ready to use our Spring AI Travel Agent and put it to the test. In the next section, you will learn how to make use of the Spring AI Agent from Claude Desktop.

Invoking Spring AI agents using an MCP client

Any old MCP Client will do, but in this particular case, we are going to see how to configure Anthropic's Claude Desktop to invoke our custom Spring AI Agent.

If you remember from the last section, our custom Spring AI Agent was a Travel Agent capable of making travel arrangements. We used Google Travel for our travel provider implementation.

> **NOTE**
>
> Make sure you have already installed the Claude Desktop app from claude.com/download as instructed in the Technical Requirements section at the beginning of this chapter.

First, we need to build the `agent-example` from the `spring-ai-examples` repository. Execute the following commands:

```
spring-ai-examples $ cd agent-example
spring-ai-examples $ mvn clean package
```

You can skip the tests by specifying the `-DskipTests` option to the `mvn` command. Once the `agent-example` has been successfully built, navigate back to the `spring-ai-examples` directory. For convenience, I have created a Bash shell script to copy the `travel-agent-0.1.0-SNAPSHOT.jar` to the user home directory on your system. Simply run the following command:

```
spring-ai-examples $ bin/copyTravelAgentToUserHome.sh
```

You should see something similar to the following:

```
spring-ai-examples $ ls -l ~/
...
-rw-r--r--    1 jxblum  staff  28348879 May  1 18:38 travel-agent-0.1.0-SNAPSHOT.jar
...
```

This `travel-agent-0.1.0-SNAPSHOT.jar` is an executable Spring Boot JAR.

Now, we need to configure Claude Desktop. Run the Claude Desktop application on your computer. Then navigate to **Settings > Developer**. On the Developer screen, you will see the **"Edit Config"** button. Clicking this button navigates you to the `claude_desktop_config.json` file. On Mac, this configuration file is located in `$USER/Library/Application Support/Claude`.

When you initially open the file after a fresh install, the configuration will simply define your preferences. For existing users, you may have already altered your configuration depending on what Tools and other components you have installed. In the `spring-ai-examples/agent-example/travel-agent/src/main/resources` directory you will find a `claude-deskstop-config.json` template file. The contents of this file appear as follows:

```
{
  "mcpServers": {
    "spring-ai-mcp-travel-agent": {
      "command": "java",
      "args": [
        "-Dspring.main.web-application-type=none",
        "-Dspring.ai.mcp.server.stdio=true",
        "-DUSER_HOME=$HOME",
        "-jar",
        "$HOME/travel-agent-0.1.0-SNAPSHOT.jar"
      ],
      "env" : {
        "HOME": "/Users/jxblum"
      }
    }
  }
}
```

You must manually expand the highlighted `$HOME` environment variable in the template above. You will copy the expanded template into Claude Desktop's `claude_desktop_configu.json` file, under `preferences`, as follows:

```
{
 "preferences": {
  "coworkScheduledTasksEnabled": false,
  "ccdScheduledTasksEnabled": false,
  "sidebarMode": "chat",
  "coworkWebSearchEnabled": true,
  "floatingAtollActive": true
 },
 "mcpServers": {
  "spring-ai-mcp-travel-agent": {
   "command": "java",
   "args": [
    "-Dspring.main.web-application-type=none",
    "-Dspring.ai.mcp.server.stdio=true",
    "-DUSER_HOME=<path-to-your-system-home-directory>",
    "-jar",
    "<path-to-your-system-home-directory>/travel-agent-0.1.0-SNAPSHOT.jar"
   ],
   "env": {
    "HOME": "/Users/jxblum"
   }
  }
```

```
  }
}
```

Make sure to add "," after the `preferences` JSON configuration to separate the `mcpServers` configuration. In my configuration, I am not using the Spring Web MVC MCP server as I am simply using the STDIO, which is evident from the `spring.ai.mcp.server.stdio` JVM System property. Make sure your home directory to the JAR file, `USER_HOME` JVM System property, and the `HOME` environment variable are set accordingly.

Now, simply restart the Claude Desktop application. If everything is configured correctly, Claude Desktop should have found and started the Spring AI, Spring Boot Travel Agent application and registered the available Tools. A successful Agent registration and start appears as follows, as seen in Claude Desktop:

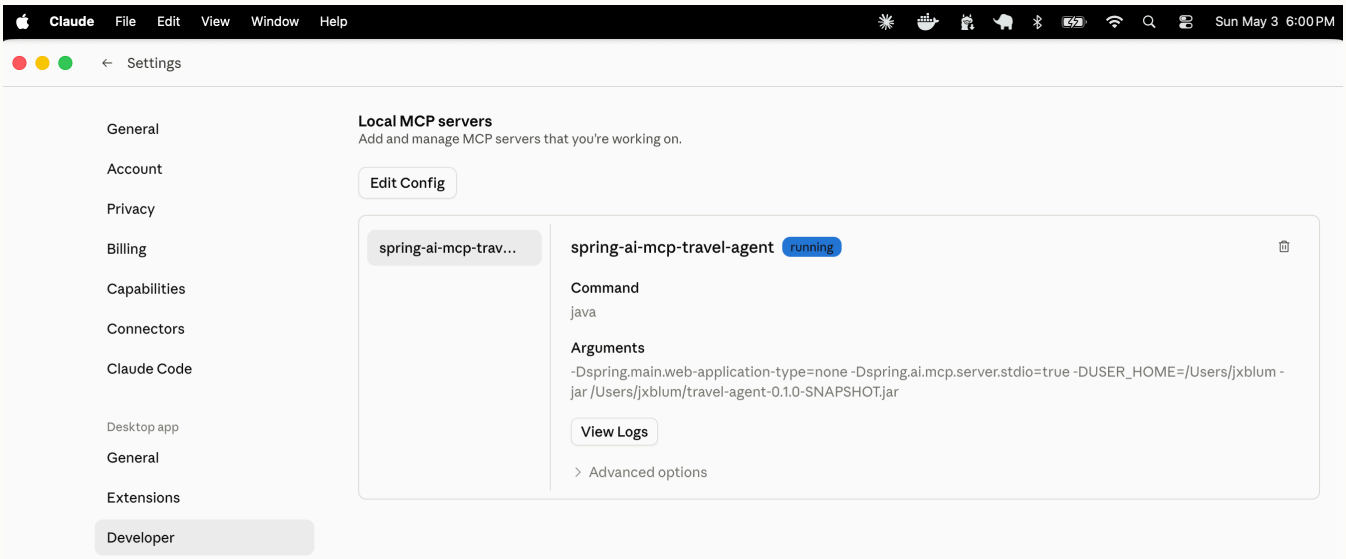


Figure 8.12: Spring AI Travel Agent registered in Claude Desktop

You can even ask Claude if the Spring AI Travel Agent is running and available for use, as seen in *Figure 8.12*:

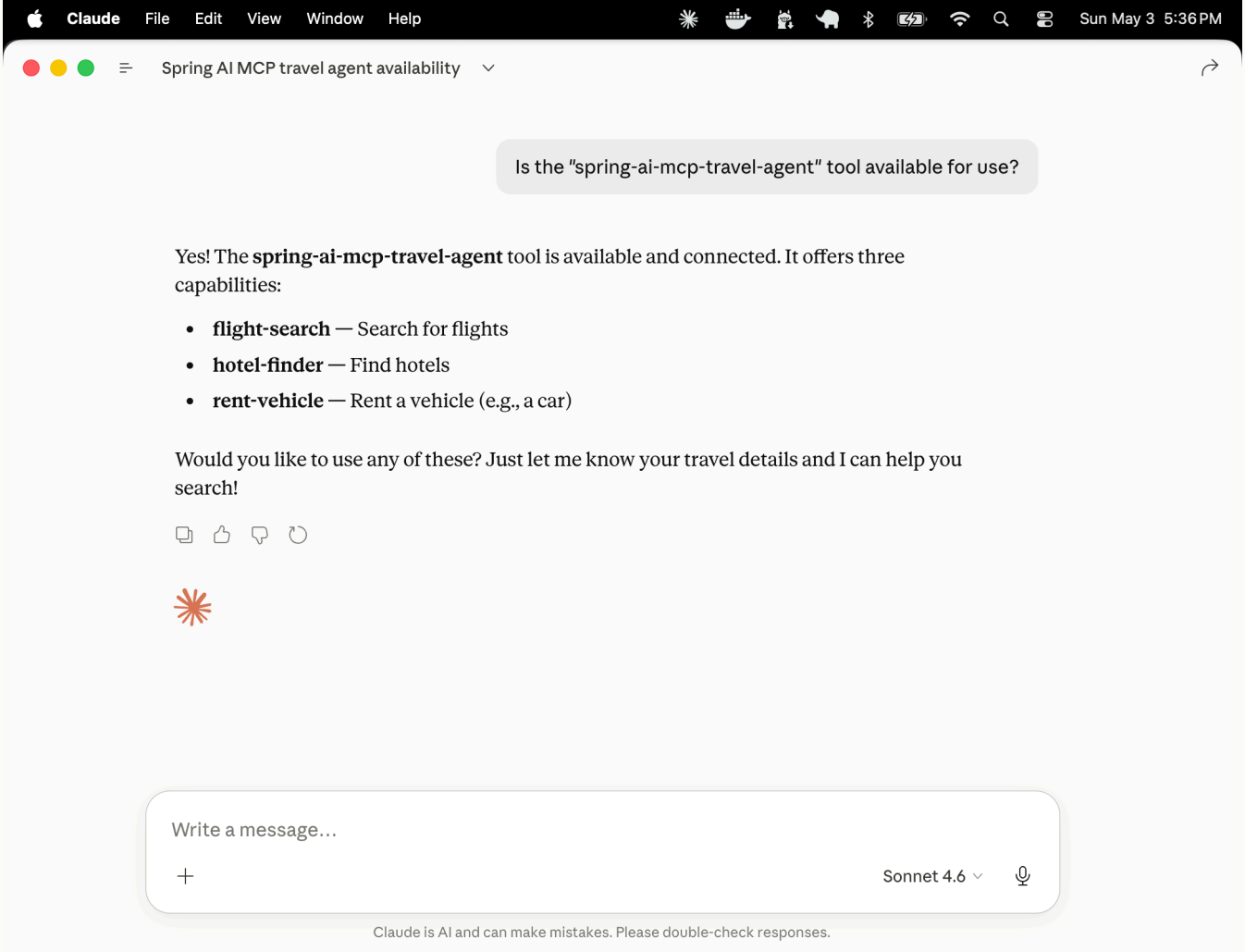


Figure 8.13: Prompting Claude about Spring AI Travel Agent availability

As you can see above, Claude even identified the available Tools along with providing their description. Now, you can run a search for flights. Simply type in a prompt with the necessary search criteria and let 'er rip:

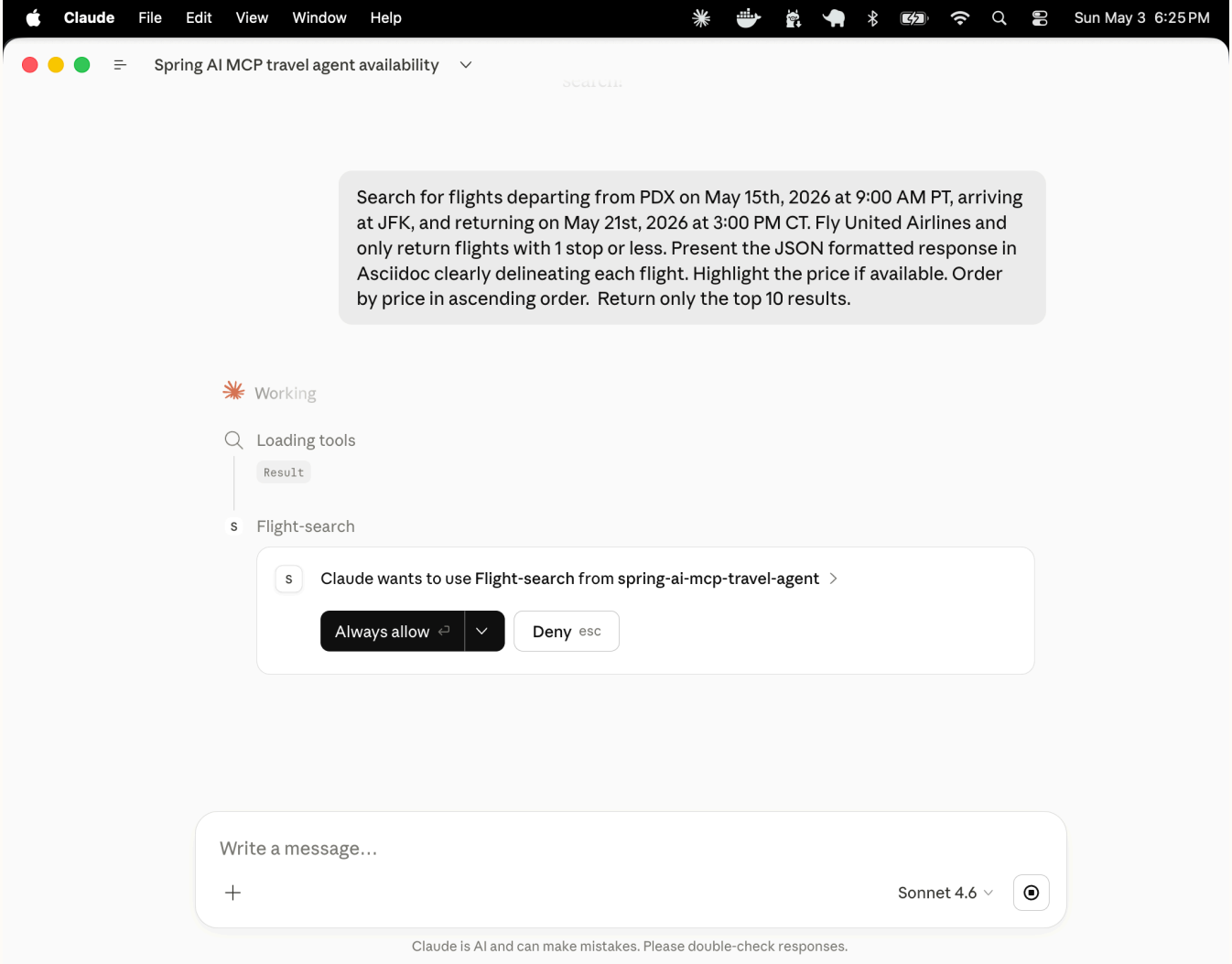


Figure 8.14: Running a flight search

Within a short few seconds, Claude will invoke the appropriate Tool, in this case, "*flight-search*," aggregate the results and return them in Asciidoc format as instructed. The result is shown below:

Of course, the Asciidoc format is not very user-friendly, at least not for the non-writing average user, but it is a flexible format that can be converted into other formats, such as HTML. So, I did just that: I instructed Claude to convert the Asciidoc into HTML, and it did a rather nice job using my "first-shot" prompt of "*Please reformat the result as HTML*." The result of the conversion can be seen below:

Just bonkers! That is a "real" Flight. We are going places now!

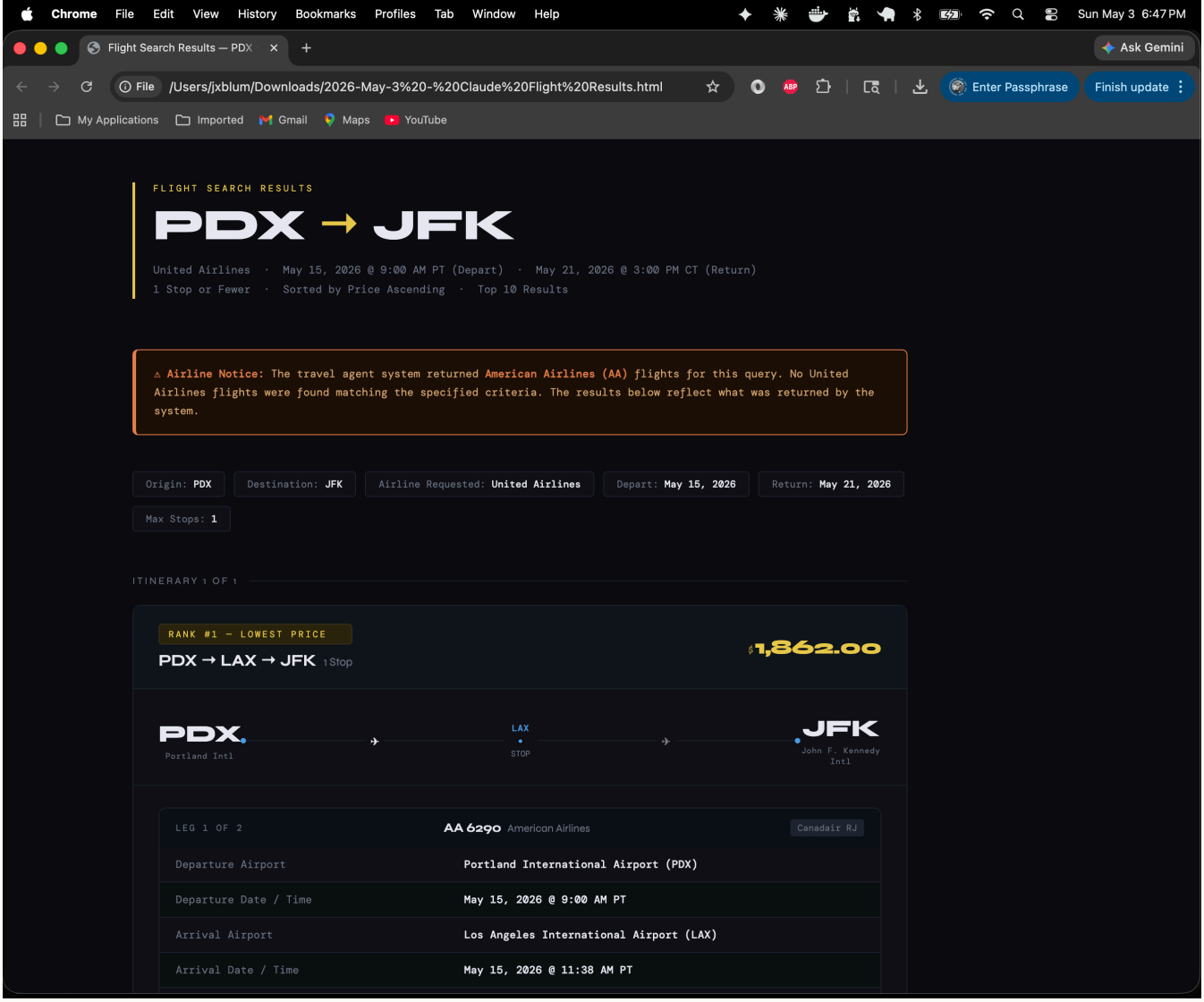


Figure 8.15: Flight Search Results presented as HTML

Well, ok, maybe not just yet. I did not implement the capability to actually reserve a flight, or even a hotel for that matter. This is just a modest example. Therefore, I am not so bold as to handle and process payment information. However, as you can imagine, this is entirely possible, also with the help of AI, and you should already be thinking about Spring Security.

There you have it. A fully functional, Spring AI (Travel) Agent using MCP and (Function) Tool calling to make travel arrangements for flight and hotel. The secure payments could be handled by a downstream AI agent waiting to make a reservation after the first AI Agent presents the travel options. You can refer to the opening sections in this chapter for choosing the right Agentic AI workflow for your use case. In this case, either a sequential or parallel workflow could be used. The "sky" is the limit. Pun intended.

Next up, I briefly mention a few other Agent AI frameworks available for the JVM platform.

Building AI agents with Embabel, Koog and LangChain4j

By using Spring AI's elegant programming model, building Agentic AI systems containing one or more Agents, either standalone or composed as part of a sophisticated workflow, is a very straightforward process. However, Spring AI is not the only game in town.

What impresses me most is how mature Spring AI and the rest of the JVM ecosystem are at handling all manners of AI use cases, not just Agentic AI. This happened in a relatively short period of time. While Python may have had a head start, there really is no decisive advantage for using Python over Java, Kotlin, or Groovy on the JVM. In fact, the maturity and breadth of JVM-based technologies is unmatched.

In addition to generative and agentic AI use cases, JVM-based technologies can also be used to build and train models. For such tasks, Deeplearning4j and Google's TensorFlow for Java are great starting points.

Go to deeplearning4j.konduit.ai for a deep dive on using Deeplearning4j to build and train custom AI models. Or, head over to github.com/tensorflow/java to learn more about how TensorFlow for Java can be used to build, train, and run machine learning models. You can also get an overview of TensorFlow at tensorflow.org/learn.

However, when it comes to Agentic AI, Embabel is one of the more recent and interesting developments in the space. For starters, Embabel was founded by the creator of the Spring Framework himself, Dr. Rod Johnson. At the time of this writing, Embabel has been publicly available since May 2025. Around that time, Dr. Johnson published an article on the Embabel Agent Framework through Medium at medium.com/@springrod/embabel-a-new-agent-platform-for-the-jvm-1c83402e0014.

In a nutshell, Embabel's core concepts center around Actions, Goals, Conditions, and Domain Models. Actions are the steps taken by an Agent to achieve a Goal, or specific, explainable outcome. It uses Conditions for evaluation during planning and Conditions can be reassessed after each Action to dynamically adapt to changes at runtime. This enables the system to replan and adjust the Actions taken to achieve a Goal. To effectively manage state and inform Actions, Goals, and Conditions, strongly typed Domain Objects are used to pass information. When combined, this forms the GOAP model, or Goal-Oriented Action Plan. Basically, Embabel creates a plan to sequence a series of Actions that achieve a particular Goal.

The process used to execute Actions to achieve the Goal is effectively an OODA loop, or Observe, Orient, Decide, and Act Loop. This loop repeats until the Goal is achieved, or the system, such as a Condition, determines the Goal cannot be reached or is too risky, etc.

By using strongly-typed Domain Objects, the Embabel framework coined another new term known as DICE, or Domain Integrated Context Engineering. In our travel agent above, this would be similar to the Travel API domain model types, such as `FlightSearchRequest`, `Flight`, `Airline`, `Airport`, `Departure`, `Arrival`, and so on.

While a full and fair treatment of the Embabel framework is well beyond the scope of this book, you can refer to the documentation to see the framework in action. See the concrete and complete example at docs.embabel.com/embabel-agent/guide/0.3.5/. Personally, I am very excited to learn more and maybe contribute to this project in the near future.

Embabel 0.3.5 is the latest version at the time of this writing. See published releases at github.com/embabel/embabel-agent/releases for the most current version.

One last interesting tidbit about Embabel is that it is written in Kotlin, which brings me to Koog. Koog, developed by JetBrains, is also the creator of the Kotlin programming language and my favorite IDE of all time, IntelliJ IDEA. JetBrains, provides many great tools! I have been a proud user since 2003.

Koog is another Kotlin-based Agent framework. One of Koog's many strengths is its ability to develop AI Agents using idiomatic Kotlin and then deploy the Agents across many platforms, including the JVM, JS, WasmJS, Android, and iOS. Another fine example of write once, then build and run anywhere.

You can learn more about Koog on JetBrains' website at jetbrains.com/koog/. Afterwards, venture over to the Koog GitHub repository and see the Quickstart example at github.com/JetBrains/koog#quickstart-example.

Finally, this brings us to LangChain4j. Unlike Spring AI, LangChain4j was based on the original Python LangChain library used to integrate AI into Java/JVM-based systems. However, like Spring AI, LangChain4j is a full-featured AI framework that can be used for many different generative AI use cases as well as Agentic AI-based workflows.

As a simple example, we might implement our Travel Agent, and in particular the "flight-search" Tool, using LangChain4j as follows. First we define the interface for our Travel Agent, like so:

```
interface TravelAgent {
  @UserMessage("""
    Search for flights departing from {departureAirport}
    on {departureDateTime}, arriving at {arrivalAirport},
    and returning on {returnDateTime}. Fly {airline}
    and only return flights with {stopCount} stop or
    less. Present the JSON formatted response in Asciidoc
```

```
        clearly delineating each flight. Highlight the price
        if available. Order by price in ascending order.
        Return only the top {resultCount} results.
    """)
    @Agent(name = "flight-search", description = "Search for flights", outputKey = "flights")
    List<Flight> searchFlights(
        @V("departureAirport") String departureAirport,
        @V("departureDateTime") ZonedDateTime departureDateTime,
        @V("returnDateTime") ZonedDateTime returnDateTime,
        @V("airline") String airline,
        @V("stopCount") int stops,
        @V("resultCount") int numberOfResults
    );
}
```

NOTE This is just a preview. This code snippet is not complete, and I did not attempt to build or run it.

After the Agent interface is defined, we can then construct an Agent object using LangChain4j's `AgentServices` factory class, as shown below:

```
TravelAgent travelAgent = AgentServices
    .agentBuilder(TravelAgent.class)
    .chatModel(openAiChatModel)
    .outputKey("flights")
    .build();
// Use travelAgent to search for flights
```

Once we have acquired an instance of a `TravelAgent`, it can be used to conduct searches for available flights. The `outputKey` attribute of the `@Agent` annotation can be used to link outputs from one Agent to the input of another Agent in a complex, multi-Agent workflow.

With just this simple example, it already provides a decent overview for quickly building AI Agents with LangChain4j. You can learn more about building Agents and Agentic AI systems using LangChain4j from the source at docs.langchain4j.dev/tutorials/agents.

Your choice of Spring AI, LangChain4j, Embabel, and Koog is a matter of preference. If Spring is a key component in your application system architecture, then Spring AI is the clear choice here. Integration with other Spring projects is simply unmatched. For me, it is all Spring AI.

Let's quickly recap what we learned before heading into the last chapter.

Summary

In previous chapters, we covered in-depth material on how to use Spring AI for complex generative AI use cases, such as Shazam. In this chapter, I provide a comprehensive treatment of building Agentic AI systems using Spring AI.

We saw how different Agentic AI workflows can be combined to form more sophisticated workflows that can be used to orchestrate Agents in a controlled and precise manner. You learned about Sequential, Parallel, Routing and Evaluator-Optimizer Workflows to maximize the collective power of multiple AI agents working in tandem without sacrificing quality.

We then put Spring AI's Agentic capabilities, such as retrieval and Tool calling, to work by implementing a live, real-time Travel Agent. This Agent pulled available travel data from Google Travel and made the functionality of searching for flights and hotels accessible as Tools through the MCP that the AI model could identify and present to the user.

Rather than build a custom user interface for this application, we simply made the Agent available to MCP clients, such as Claude Desktop. This is very appealing, especially during development.

Finally, we concluded by providing a quick overview of other Agent-based Frameworks, such as Embabel, Koog and LangChain4j, in the JVM ecosystem.

As Dr. Rod Johnson so elegantly stated in his article covering Embabel, it is an exciting time to be a JVM developer, and particularly an AI engineer building enterprise-grade, Generative and Agentic AI systems using Java or Kotlin, along with Spring AI or your framework of choice!

What will you build?

We now head into the last chapter where I will share my final thoughts on AI and what the future might hold.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns—*AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at https://packt.link/8Oz6Y or scan the QR code below.



Previous chapter
‹ Chapter 7: Become a Spring AI Power User

Next chapter
Chapter 9: Demystifying the Impact of AI ›